



Curso ROS2 básico: Python

Aula 01 — Estrutura de Projeto no ROS 2: Workspace, Pacotes e Compilação

Objetivo

Compreender a estrutura de um projeto ROS 2, incluindo workspace, pacotes, arquivos de metadados, configuração e compilação usando `colcon`. Ao final, o estudante será capaz de criar um pacote funcional e entendê-lo em profundidade.

1. O que é um Workspace?

O **workspace** é o diretório onde você organiza e desenvolve seus pacotes no ROS 2. Ele é composto por três principais pastas:

```
ros2_ws/  
├── src/      # Código-fonte dos pacotes  
├── build/    # Arquivos intermediários de compilação (gerado pelo colcon)  
└── install/  # Binários instalados e ambiente final (gerado pelo colcon)
```

Criando um workspace:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

2. O que são Pacotes ROS 2?

Um **pacote ROS 2** é uma unidade que agrupa:

- nós
- mensagens
- serviços
- arquivos de configuração
- scripts e dependências

Existem dois tipos principais:

- `ament_python` (pacotes em Python)
- `ament_cmake` (pacotes em C++)

Criando um pacote em Python:

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python --dependencies rclpy meu_primeiro_pacote
```

O parâmetro `--dependencies` é usado para declarar dependências que o pacote precisa. No exemplo acima, usamos `rclpy`, que é essencial para desenvolver nós em Python no ROS 2.

Você pode adicionar mais de uma dependência separando por espaço:

```
ros2 pkg create --build-type ament_python <nome_do_pacote> --dependencies rclpy rclpy
```

Resultado:

```
meu_primeiro_pacote/
├─ package.xml
├─ setup.py
├─ setup.cfg
├─ resource/
└─ meu_primeiro_pacote/
    └─ __init__.py
```

3. Arquivos Importantes

```
package.xml
```

Arquivo de metadados com informações do pacote:

- nome, versão, descrição
- autor, licença
- dependências

```
setup.py e setup.cfg
```

Definem a instalação do pacote Python.

```
CMakeLists.txt
```

Presente apenas em pacotes C++ (ament_cmake).

4. O que é o `colcon`?

O `colcon` é a ferramenta de compilação oficial do ROS 2. Ele substitui o antigo `catkin_make` do ROS 1.

- Constrói múltiplos pacotes em ordem correta
- Gera as pastas `build/`, `install/` e `log/`
- Suporta C++, Python e outras linguagens

Compilando:

```
cd ~/ros2_ws
colcon build
```

Esse comando compila todos os pacotes presentes na pasta `src/` do workspace.

Compilando um pacote específico:

```
colcon build --packages-select meu_primeiro_pacote
```

Substitua `meu_primeiro_pacote` pelo nome do seu pacote. Esse comando compila apenas o pacote indicado, útil para agilizar testes e evitar recompilar tudo.

```
bash cd ~/ros2_ws colcon build
```

Compilando um pacote específico:

```
colcon build --packages-select meu_primeiro_pacote
```

5. O que faz o `source` ?

O comando `source` é utilizado para **carregar o ambiente ROS 2** no terminal, permitindo que os pacotes compilados sejam encontrados e executados.

Usando:

```
source install/setup.bash
```

Tornando permanente:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

6. Comandos de Pacotes

Criar um pacote:

```
ros2 pkg create --build-type ament_python meu_pacote
```

Listar pacotes:

```
ros2 pkg list  
ros2 pkg list | grep meu_pacote
```

Localização do pacote instalado:

```
ros2 pkg prefix meu_pacote
```

Ver executáveis de um pacote:

```
ros2 pkg executables meu_pacote
```

Executar um nó:

```
ros2 run meu_pacote nome_do_no
```

7. Resumo Visual

- Estrutura de um workspace

- Estrutura de um pacote Python
 - Relação entre `src`, `build`, `install`, e `O source`
 - Linha de tempo do processo: criação → compilação → execução
-

Próximas aulas: criaremos nossos primeiros nós e exploraremos os tipos de comunicação entre eles usando os pacotes que construímos aqui.

MetaBee | Educação, Inovação e Robótica